

RNA Pairing Pattern Recognition Using Stochastic Context-Free Grammars and Evolutionary History

Ali Ahmadi Esfidi

September 16, 2026

Abstract

We predict the consensus secondary structure of an RNA alignment, pseudoknots included, by combining the Knudsen–Hein (KH-99) stochastic context-free grammar with a phylogenetic model of single and paired columns. A *Gap-Bracket CYK* parser scores each base pair with two ratios, one for helix starts and one for stacked pairs, and runs twice: the first pass finds the nested structure and the second searches the remaining columns for crossing pairs, with a ratio applied for every crossing. The method is implemented as a small NumPy package. The parser’s recursions are checked against exhaustive search, and predicting an alignment takes 10–100 ms, from 50 to over 10,000 times faster than our original implementation. On eight held-out Rfam families, the two-pass parser reaches a base-pair F1 of 0.804, compared with 0.669 for plain KH-99 CYK (bootstrap 95% CI of the gain: 0.007–0.331).

1 Introduction

RNA molecules take part in most major biological processes, from regulating gene expression to catalysing reactions. Their function depends on base pairing, which forms helical stems and loops. Many prediction methods ignore crossing interactions (pseudoknots), which limits their accuracy on structured RNAs.

This report extends the KH-99 [6] and Pfold [7] approach, which combines a structure grammar with a phylogenetic model of the alignment, in two ways. First, the CYK parser gets ratios that favour long helices. Second, a second parsing pass revisits the columns left unpaired and recovers crossing pairs. We describe the model precisely, fix several issues in our earlier implementation, and evaluate on held-out Rfam families against the KH-99 baseline, with confidence intervals.

2 Related Work

KH-99 [6] combined SCFGs with evolutionary information. Pfold [7] and PPfold [11] refined the approach with explicit phylogenetic trees, and PPfold parallelised it. Pseudoknots are outside the reach of context-free grammars; Stochastic Multiple Context-Free Grammars [5] and Pair Stochastic Tree Adjoining Grammars [8] model them directly, at a higher polynomial cost. Knotify [1] detects pseudoknots with syntactic pattern recognition. Deep-learning models such as E2Efold [2] and SPOT-RNA [10] predict pairing matrices directly, including crossing pairs. Our method stays within the cubic CYK framework and handles pseudoknots with a second pass.

3 Method

The input is an alignment of N sequences with n columns; the output is a consensus dot-bracket structure in which $()$ marks nested pairs and $[]$ crossing pairs.

3.1 Structure grammar

We use the KH-99 grammar in Chomsky normal form:

$$\begin{aligned} S &\rightarrow L S \mid \$M E \mid s & L &\rightarrow \$M E \mid s \\ F &\rightarrow \$M E \mid L S & \$M &\rightarrow B F \\ B &\rightarrow d & E &\rightarrow d \end{aligned}$$

S generates a loop region of one or more elements, and L one element, which is either an unpaired base s or a helix. $\$M E$ generates a base pair $d F d$. F , the interior of a pair, is either another pair (stacking) or a loop of at least two elements. Every nested structure has exactly one derivation, so the maximum-likelihood rule probabilities are the relative frequencies of the rules used by the training structures.

3.2 Evolutionary model

Unpaired columns evolve under a 4×4 rate matrix R , and paired columns under a 16×16 rate matrix over

dinucleotides [6]. Both are estimated from Rfam seed families that have trees. Each family has the same total weight, split evenly over its sequences. For ordered pairs of sequences with at least 85% identity, we count substitutions c_{XY} in unpaired columns and $c_{XY,X'Y'}$ in paired columns, symmetrised so that $XY \rightarrow X'Y'$ also counts $YX \rightarrow Y'X'$. We then set

$$R_{XY} = \frac{c_{XY}}{P_s P_X \sum_p t_p N_p}, \quad R_{XX} = - \sum_{Y \neq X} R_{XY},$$

where t_p is the tree distance and N_p the number of comparable columns of pair p , P_X the frequency of X , and P_s the fraction of unpaired positions. The paired matrix uses $2/P_p$ in place of $1/P_s$, since a paired column holds two positions. Counts from one sequence are averaged over its similar partners. Ambiguous IU-PAC symbols are spread uniformly over the compatible nucleotides.

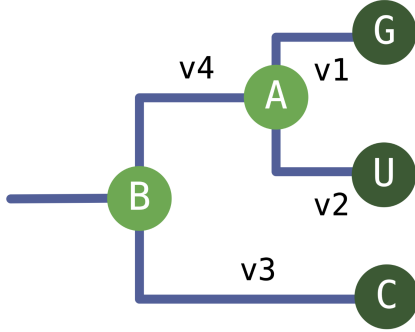


Figure 1: A tree with nucleotides G, U and C at the tips; the interior states A and B are unknown.

For the tree in Figure 1, the probability of the tip states together with interior states A, B is [3]

$$P(GUC, AB) = \pi_A p_{AG}(v_1) p_{AU}(v_2) p_{AB}(v_4) p_{BC}(v_3),$$

where $p_{XY}(t) = [e^{Rt}]_{XY}$. The likelihood of the column sums over all interior states. Felsenstein's pruning algorithm computes this sum in time linear in the number of nodes. A gap is treated as an unknown nucleotide (a tip vector of all ones) [7]. We compute $\log P_{\text{single}}(i)$ for all columns and $\log P_{\text{pair}}(i, j)$ for all n^2 column pairs at once, as arrays of shape $n \times 4$ and $n \times n \times 16$. Every node is rescaled, so large alignments do not underflow.

Tree. The alignment tree has a neighbour-joining topology [9] built from Jukes–Cantor distances. Its branch lengths are set by maximum likelihood under R : two sweeps of golden-section search per branch, using inside and outside partial likelihoods. The tree is then rooted at its midpoint. A PhyML tree [4] can be used instead.

3.3 Gap-Bracket CYK

Extending the grammar with the column evidence gives the Viterbi recursions below. All terms are log-probabilities, q denotes rule probabilities, and ρ_s , ρ_a , ρ_f are the *start*, *accelerate* and *flag* ratios.

$$\begin{aligned} \text{Pair}(i, j) &= M(i, j-1) + \log P_{\text{pair}}(i, j) + \chi(i, j) \log \rho_f \\ M(i, j) &= \log q_{\$M \rightarrow BF} + \max (F_{\text{st}}(i+1, j) + \log \rho_a, \\ &\quad F_{\text{lp}}(i+1, j) + \log \rho_s) \\ F_{\text{st}}(i, j) &= \log q_{F \rightarrow \$ME} + \text{Pair}(i, j) \\ F_{\text{lp}}(i, j) &= \log q_{F \rightarrow LS} + \max_k L(i, k) + S(k+1, j) \\ L(i, j) &= \log q_{L \rightarrow \$ME} + \text{Pair}(i, j) \quad (i < j) \\ S(i, j) &= \max (\log q_{S \rightarrow \$ME} + \text{Pair}(i, j), \\ &\quad \log q_{S \rightarrow LS} + \max_k L(i, k) + S(k+1, j)) \end{aligned}$$

For a single column, $S(i, i) = \log q_{S \rightarrow s} + \log P_{\text{single}}(i)$, and likewise for L . The start ratio ρ_s applies to a pair that closes a loop, i.e. the innermost pair of a helix. The accelerate ratio ρ_a applies to a pair stacked on another pair, so $\rho_a > \rho_s$ favours long, uninterrupted helices. Because F is split into its stacked and loop derivations, the ratios are applied exactly. The earlier implementation kept one F cell, and the ratio depended on whichever derivation happened to score best there. $\chi(i, j)$ counts the first-pass pairs that cross (i, j) .

Each span length is filled with one vectorised array operation, so a pass costs $O(n^3)$ arithmetic but only $O(n)$ Python iterations. The recursions and traceback are tested against exhaustive enumeration of all structures on random inputs.

First pass. The parser runs on all columns ($\chi = 0$) and returns the nested structure, written with $()$.

Second pass. The first-pass pairs are removed. The parser runs again on the remaining columns, with its own ρ_s , ρ_a and the crossing weight ρ_f , and the pairs it finds are written with $[]$. Repeating the procedure would find deeper pseudoknots.

4 Parameters

4.1 Data

Three Rfam families with a range of observed mutations define the evolutionary model and the grammar: RF00001 (5S rRNA), RF00005 (tRNA) and RF03000 (LOOT RNA motif). Three further families (RF01704, RF02035, RF03054) were tried early on and not used. The ratios are tuned on eight small validation families, and the method is evaluated on eight test families (Table 1). RF01737 appears in both sets, with different sequences, so we also report test scores without

it. One validation reference, RF01722, contained an unmatched ‘(’, which we treat as unpaired.

Set	Families	Seqs	Cols
Training	RF00001, RF00005, RF03000	368–954	118–230
Validation	RF01862, RF01834, RF01077, RF01090, RF01080, RF01722, RF03537, RF01737	4–10	32–70
Test	RF00556, RF00499, RF01084, RF03004, RF01089, RF01093, RF01072, RF01737	3–25	33–191

Table 1: Rfam families per set.

4.2 Evolutionary parameters

GC/CG pairs dominate stems (Table 2), in line with their high stacking energy. Transitions ($A \leftrightarrow G$, $C \leftrightarrow U$) are faster than transversions (Table 3). In paired columns, changes that keep a pair canonical, such as $UG \rightarrow CG$ or $AU \rightarrow GC$, are frequent, while changes that need two transversions are rare (Table 4). Our re-implementation reproduces the parameters of the original notebooks to a relative difference of 10^{-4} . The only difference is how ambiguous IUPAC symbols are handled.

Paired bases		Single bases	
GC/CG	53.2%	A	37.2%
AU/UA	30.8%	C	19.0%
GU/UG	10.1%	G	18.7%
Others	5.9%	U	25.1%
Of positions	61.1%	Of positions	38.9%

Table 2: Base frequencies in the three training families.

X / Y	A	C	G	U
A	−0.44	0.12	0.17	0.15
C	0.24	−0.77	0.12	0.41
G	0.33	0.12	−0.62	0.17
U	0.22	0.32	0.13	−0.66

Table 3: Single-base rate matrix R .

X / Y	AU	UA	GC	CG	UG	GU
AU	−1.66	0.28	0.68	0.21	0.03	0.25
UA	0.28	−1.66	0.21	0.68	0.25	0.03
GC	0.37	0.11	−1.14	0.24	0.03	0.26
CG	0.11	0.37	0.24	−1.14	0.26	0.03
UG	0.10	0.76	0.16	1.47	−2.84	0.03
GU	0.76	0.10	1.47	0.16	0.03	−2.84

Table 4: Entries of the paired rate matrix between canonical pairs (the diagonal also includes rates to non-canonical pairs).

4.3 Grammar parameters

Relative rule frequencies in the nested part of the training structures:

$$\begin{aligned}
 S &\rightarrow LS \text{ (88.4\%)} \mid \$ME \text{ (2.3\%)} \mid s \text{ (9.3\%)} \\
 F &\rightarrow \$ME \text{ (61.6\%)} \mid LS \text{ (38.4\%)} \\
 L &\rightarrow \$ME \text{ (8.6\%)} \mid s \text{ (91.4\%)} \\
 \$M &\rightarrow BF \text{ (100\%)} \quad B, E \rightarrow d \text{ (100\%)}
 \end{aligned}$$

The original notebooks estimated these with inside–outside on strings of paired/unpaired symbols, which gave similar values (88.6/1.9/9.5, 62.5/37.5, 8.7/91.3%). That estimate does not know which bases pair with which, so we use the exact counts instead.

4.4 Gap-Bracket ratios

The ratios were chosen by grid search on the validation set, maximising pooled base-pair F1. The grid covered $\{0.25, 0.35, 0.5, 0.7, 1, 1.4, 2, 2.8, 4\}$ for ρ_s, ρ_a and $\{0.25, \dots, 2\}$ for ρ_f . The first pass was tuned alone, then the second pass on top of it, and ties went to the setting closest to 1. The whole search takes about 10 seconds. The genetic algorithm of the original notebooks needed an estimated several hours, because every evaluation re-ran PhyML and a dictionary-based parser.

	Start ρ_s	Accelerate ρ_a	Flag ρ_f
<i>Grid search (this report, default)</i>			
1st pass	0.35	2.00	–
2nd pass	1.00	1.40	1.00
<i>Genetic algorithm (2025 notebooks)</i>			
1st pass	0.19	1.48	–
2nd pass	0.88	1.78	0.98

Table 5: Gap-Bracket ratios. Both searches find $\rho_s < 1 < \rho_a$ in the first pass: helices are cheap to extend and expensive to start.

5 Results

The main measure is pooled base-pair F1 over the families of a set. For continuity with the 2025 report we also give the *weighted* score used there: pair F1 and unpaired-column F1 averaged with weights $2 \times (\text{reference pairs})$ and $(\text{reference unpaired columns})$. Confidence intervals come from resampling families with replacement (10,000 resamples).

Model	P	R	F1	Wtd.
<i>Validation (8 families, used for tuning)</i>				
KH-99 CYK	.820	.701	.756	.780
Gap-Bracket, 1st pass	.846	.752	.796	.814
Gap-Bracket, 2 passes	.836	.872	.854	.866
with 2025 GA ratios	.832	.846	.839	.854
<i>Test (8 held-out families)</i>				
KH-99 CYK	.748	.604	.669	.703
Gap-Bracket, 1st pass	.770	.731	.750	.750
Gap-Bracket, 2 passes	.769	.843	.804	.798
with 2025 GA ratios	.793	.858	.824	.818

Table 6: Pooled precision, recall, base-pair F1 and the legacy weighted score. KH-99 CYK is the same parser with all ratios equal to 1 and one pass.

Table 6 and Figure 2 summarise the results. On the test set, the first-pass ratios raise F1 from 0.669 to 0.750, mostly through recall: helices are no longer broken into fragments. The second pass adds 0.054 F1 (95% CI 0.006–0.116) by recovering pseudoknotted helices, and raises recall to 0.843 with almost no loss of precision. Overall, the gain over KH-99 CYK is 0.135 (CI 0.007–0.331). Without the overlapping family RF01737, test F1 is 0.650, 0.743 and 0.803 for the three models.

The genetic-algorithm ratios score lower than the grid-search ratios on validation (0.839 vs 0.854) but higher on test (0.824 vs 0.804; difference CI 0.001–0.050). Tuning five ratios on eight small families overfits somewhat. Both settings were fixed on validation data, so we keep the grid-search ratios as the default rather than choosing between them by test score. With a benchmark this small, intervals are wide, and differences of a few F1 points should not be over-interpreted.

Reproduction. With PhyML trees and the genetic-algorithm ratios, as in the original notebooks, the new implementation gives a weighted test score of 0.817. The 2025 report gave 0.824, and four of the eight families (RF00556, RF00499, RF01072, RF01737) agree to within 0.1 points. The remaining differences come from the exact application of the ratios (Section 3.3), the crossing count over the full pair span, and the exact

grammar counts.

Family	Seqs	Cols	2025 code	This work
RF01737	3	68	3.3 s	10 ms
RF01072	25	33	5.5 s	104 ms
RF00556	10	46	6.5 s	40 ms
RF01093	12	60	14.6 s	50 ms
RF03004	9	84	27.8 s	41 ms
RF00499	5	117	54.6 s	27 ms
RF01089	7	134	736 s	44 ms
RF01084	9	191	>40 min [†]	82 ms

Table 7: Wall-clock time to predict one test alignment (tree, column likelihoods and both passes) on an Apple-silicon laptop. The 2025 code ran PhyML under Rosetta. [†]Stopped after 40 minutes.

Speed. Table 7 compares running times. The original implementation wrote an extended grammar with one rule per distinct column pattern to disk, then ran a dictionary-based CYK over it, so its cost grew with both alignment length and the number of distinct columns. The new implementation computes all column likelihoods as arrays and fills each span length of the chart in a single vectorised step.

6 Discussion

Two failure modes remain.

1. **Alignments unlike the training families.** The model is estimated from only three families. RF01084 (191 columns), for example, contains long, gappy regions. There, the pair evidence is weak and the grammar prefers short local helices.
2. **Poorly conserved, gappy columns.** Gaps are unknown nucleotides, so a column that is filled in only one or two sequences can still support a pair. The structure of such a region then depends on very few sequences.

RF00499 (first 54 columns) shows the effect of the first-pass ratios. KH-99 CYK breaks the long central helix into fragments, while the Gap-Bracket parser recovers most of it:

RF00499

Ref	...(((((((((((((((((((((((((((((.....))))))..)))
KH99	...(((((((.....((((.....((((((((.....))))))....
GB	...(((((((.....((((((((((((((((((((.....))))))..)))

RF01084 (first 54 columns) shows the gap problem. The first 38 columns are present in only one of nine sequences, yet both parsers place helices there, and

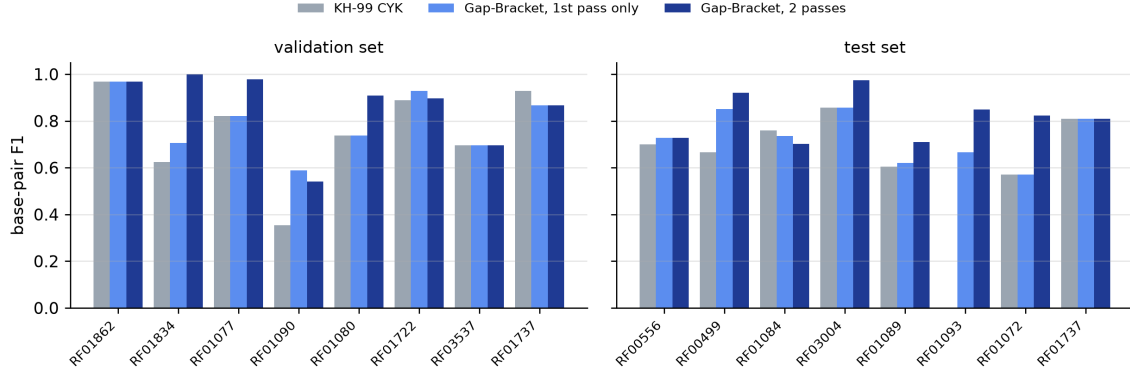


Figure 2: Base-pair F1 per family. The second pass mainly helps families with pseudoknots (RF01834, RF01077, RF01080, RF03004, RF01093, RF01072).

the Gap-Bracket parser even predicts spurious cross-pairs:

RF01084	
Ref	(((((((.....(((((((.....
KH99	(((((((.....(((((((.....
GB	(((((((.....(((((((.....

Clustering families and training a model per cluster could address the first issue. For the second, pairs could require a minimum number of sequences with bases in both columns, or the column likelihood could be weighted by how many sequences are gap-free there.

7 Conclusion

The model rests on three assumptions: stems tend to be long and adjacent; whether a pair starts or extends a helix changes its probability; and pseudoknotted helices can be found once the nested structure is fixed. The start and accelerate ratios express the first two, and the two-pass parser expresses the third. On held-out families they raise base-pair F1 from 0.669 (KH-99 CYK) to 0.804. The clean implementation makes the method fast enough to tune and evaluate in seconds. With that in place, the next steps are a larger benchmark, more training families, and loop-length-dependent grammar parameters.

References

- [1] C. Andrikos and Makris. Knotify: An efficient parallel platform for rna pseudoknot prediction using syntactic pattern recognition. *Biology*, 12:1–16, 2023.
- [2] X. Chen, Y. Li, R. Umarov, X. Gao, and L. Song. Rna secondary structure prediction by learning unrolled algorithms. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [3] J. Felsenstein. Evolutionary trees from dna sequences: A maximum likelihood approach. *Journal of Molecular Evolution*, 17:368–376, 1981.
- [4] S. Guindon, J.-F. Dufayard, V. Lefort, M. Anisimova, W. Hordijk, and O. Gascuel. New algorithms and methods to estimate maximum-likelihood phylogenies: assessing the performance of PhyML 3.0. *Systematic Biology*, 59(3):307–321, 2010.
- [5] Y. Kato and H. Seki. Stochastic multiple context-free grammar for rna pseudoknot modeling. In *Proceedings of the Eighth International Conference on Parsing Technologies*, pages 112–123. Association for Computational Linguistics, ACL, 2006.
- [6] B. Knudsen and J. Hein. RNA secondary structure prediction using stochastic context-free grammars and evolutionary history. *Bioinformatics*, 15(6):446–454, 1999.
- [7] B. Knudsen and J. Hein. Pfold: RNA secondary structure prediction using stochastic context-free grammars. *Nucleic Acids Research*, 31(13):3423–3428, 2003.
- [8] H. Matsui and Sato. Pair stochastic tree adjoining grammars for aligning and predicting pseudoknot rna structures. *Bioinformatics*, 21(11):2611–2617, 2005.
- [9] N. Saitou and M. Nei. The neighbor-joining method: a new method for reconstructing phylogenetic trees. *Molecular Biology and Evolution*, 4(4):406–425, 1987.

- [10] J. Singh and Hanson. Rna secondary structure prediction using an ensemble of two-dimensional deep neural networks and transfer learning. *Nature Communications*, 10:5407, 2019.
- [11] Z. Sükösd and Knudsen. Ppfold 3.0: fast rna secondary structure prediction using phylogeny and auxiliary data. *Bioinformatics*, 28(20):2691–2692, 2012.